MIT MANAGEMENT EXECUTIVE EDUCATION | MIT Schwarzman College of Computing

MIT SLOAN SCHOOL OF MANAGEMENT
MIT SCHWARZMAN COLLEGE OF COMPUTING

# MODULE 3 UNIT 2
# Media Set Video 2 Transcript



DELIVERED IN COLLABORATION WITH getsmarter™

# Module 3 Unit 2 Media Set Video 2 Transcript

**Generalist vs. specialized gents**

PAUL MCDONAGH-SMITH: So Tim, I think your KramaBench work reveals something really striking for organizational strategy. Multi-agent architectures with iterative planning dramatically outperformed single-agent approaches, roughly 50% versus 22% percent end-to-end accuracy. Now, of course, that's not a marginal improvement. It's a fundamentally different success rate.

For our participants, for our audience here, designing agentic AI workflows back in their organizations, what are the practical implications of this? Should companies be thinking about deploying one powerful generalist agent or orchestrating teams of specialized agents? And how does that choice map into the way we already think about organizing human teams?

TIM KRASKA: I mean, like, in the end, I think that it almost doesn't matter to be really honest, because you want to have something which works for you. Currently, I would say that these multi-agent systems is one way to organize the development of something which is more complex because, like, you cannot have a single prompt which does it all. So you need to spread it out a little bit, but you, like, you need some orchestration step. If it's one agent which shows first an orchestration, then it picks it up again and then executes the, you know, like the plan, and then it's the same agent again which uses a subplan of it.

In the end, this is kind of what's happening anyway because you have one model which is like primed or has a different prior depending on what it's supposed to do. So like, I think like the multi-agent thing is just like something to organize, like work and to develop on something bigger. Often, they have one agent at the top, which is the front, and takes all the requests and then it distributes things out to the subtask and so on. There's one big difference though or big advantage for multi-agent system and that's parallelism.

And so, we see for various tasks by now, it's just like these models take time for the inference and so often you can try out a lot of different things in parallel. And then just take the best one. Right? And so because of that, the time aspect, parallelization is a very powerful construct and therefore I believe that in the end everything will be like what some people call a multi-agent system, right, because you want to have the request come in, try to first figure out what the request is supposed to do, how to solve it, what are the different steps for it, and then there might be different alternatives and maybe you can try all the alternatives as well as certain things in parallel afterwards so that you get a faster response back in the end and a better response because you tried different things. Right?

So you see by now, like, some cases, like, thousands of agents born off at the same time to solve certain things, which was like also something unthinkable before like even a year ago.

MCDONAGH-SMITH: And think it's interesting as well where those kind of that evolution of experimentation fits into how as business leaders, how as business professionals, we're exploring our options, almost the, should we say, the selection kind of process that we go through. Then the variation and the kind of all the things that we're doing in parallel, and what are the results then that we lock into the next steps—

KRASKA: Right. And I think like to your point here, I think the harder question is like how do you define success for an agent system? It doesn't matter if it's multi-agent or single-agent, like there's a shared window probably somewhere which humans interact with or an application which humans interact with, and what does success mean, right, and how it's implemented. I don't know. The whole dev just like borns out of like a lot of other agents almost like it doesn't matter what you care about, like is it successful, but really showing success is hard. More often, you do a benchmark which is predefined, but then how well does it generalize to something outside the scope of the benchmark? And if you only use the benchmark for evaluation, I can almost guarantee you the people developing this multi-agent system, they will optimize for the benchmark.

MCDONAGH-SMITH: Exactly. Yeah. So I think you bring up a really important point for our audience as well, the vast majority of whom will have KPIs and KHIs, the health indicators, and be benchmarked against those and kind of see progress in those ways. Perhaps, you know, to be somewhat provocative, perhaps we're entering into a phase where new units of measurement, new metrics of value, new ways of actually thinking about what success actually is are coming into play, which are going to have an impact on organizational strategy or organizational theory. You know, we're entering a very exciting phase.

KRASKA: I agree. I totally agree with that.

**Error responses**

MCDONAGH-SMITH: So Tim, when an AI agent makes a consequential error in a production environment, what should the organizational response look like in your opinion? How should we be thinking about designing escalation paths and human-in-the-loop checkpoints before something goes wrong?

KRASKA: That's a really good question and I think it's like probably key to making many of these agents actually work. For like early on, I mentioned it early like previously for ML for systems, this non-index work, we actually spent a lot of time to guarantee that we get the right answers. Right? There are with agents framework, sometimes you can do something similar. You can restrain it in ways that you know what it produces back is actually correct.

To give you one example, like natural language to SQL, it's like one of the agents I've worked on for quite some time at MIT as well as at Amazon, pair definition, it can make mistakes. Right? It might pick the wrong table. It might pick the wrong join pass. They are like mistakes which can make I would argue humans make similar mistakes, but nevertheless these mistakes happen.

So there are two ways to see that. One is you live with that and just say like, okay, the human needs to have the final look at the query and then confirm it's correct or not. The other approach is like, which we also explored a lot with, is that we only allow to generate queries which we know the answer is correct. So how we did that is we had a collection of the most common queries run over, let's say, database. You ask me a question, "What is the revenue of last month?" And then it tries to find the most related known query and gives you an answer along the lines of like, "Oh, I assume you mean the revenue in America over January and here's the query for it and this is the result." Right?

And the reason why we did that is now we know that it's like it doesn't answer your question anymore. It phrases the answer back to you based on something we knew was correct and gives you the answer for the thing we know is correct as well. Right?

And so you have then a system which doesn't make mistakes anymore. The downside is now it's also not that flexible anymore because you have a restricted set of things. And now you can do something in between which is just like, oh, we only allow answers which we know are correct. The small modifications but not big ones. There's still a chance that something is wrong, but now it's more restricted. I think in many cases, can build the spectrum out. You can say like how restrictive the answer should be and with that more correct or you can do the free-form one. And now then depending on the use case, you need to pick some—

MCDONAGH-SMITH: Context.

KRASKA: On the context. Right? I would say, like, this is overall what I would recommend, like, that you figure based on the scenario where you're on the spectrum of like, uncertainty you want to be. Right? And as more uncertainty you allow, the more powerful they also become but the higher the risk is.

The other thing which I think is very important is there's like this new idea of like formal methods or automatic reasoning, to really verify answers, post mortem or have like conditions which you can check. For example, if you're a reimbursement agent, you might want to say like, "Okay, reimbursements under $500. They are just fine. If it's above $500, you should get additional approval." Right? And, you can implement that in a way that before the money is issued, the function which has to be called in order to, like, distribute the money, trigger the money transfer, that one is checked in that way. Right?

So, like, this has a hardcoded thing in there which is not LLM-generated so you know it will be executed every single time to guarantee that this escalation path exists. There are other theories out there that you can do like more like compensation actions. So the agent does something, but there should always be a way to essentially roll back. This is more early stage. I don't really know if it will work in practice, but there are methods out there right now which we have at our disposal already to make these agents more safe in certain scenarios.

SPEAKER: Did you understand all the concepts covered in this video? If you'd like to review any of the content, please click on the chapter menu.